

Nama: Muhammad Nelwan Fakhri Github Repo: <https://github.com/shrxxxk/Multimedia-Audio-Processing>

2.1 Soal 1: Rekaman dan Analisis Suara Multi-Level

- Rekamlah suara Anda sendiri selama 25 detik dimana Anda membaca sebuah teks berita.
- Dalam 25 detik rekaman tersebut, Anda harus merekam: – 5 detik pertama: suara sangat pelan dan berbisik – 5 detik kedua: suara normal – 5 detik ketiga: suara keras – 5 detik keempat: suara cempreng (dibuat-buat cempreng) – 5 detik terakhir: suara berteriak
- Rekam dalam format WAV (atau konversikan ke WAV sebelum dimuat ke notebook).
- Visualisasikan waveform dan spektrogram dari rekaman suara Anda.
- Sertakan penjelasan singkat mengenai hasil visualisasi tersebut.
- Lakukan resampling pada file audio Anda kemudian bandingkan kualitas dan durasinya

```
In [8]: import librosa, librosa.display
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from scipy.signal import butter, filtfilt
from IPython.display import Audio
import soundfile as sf
import pyloudnorm as pyn

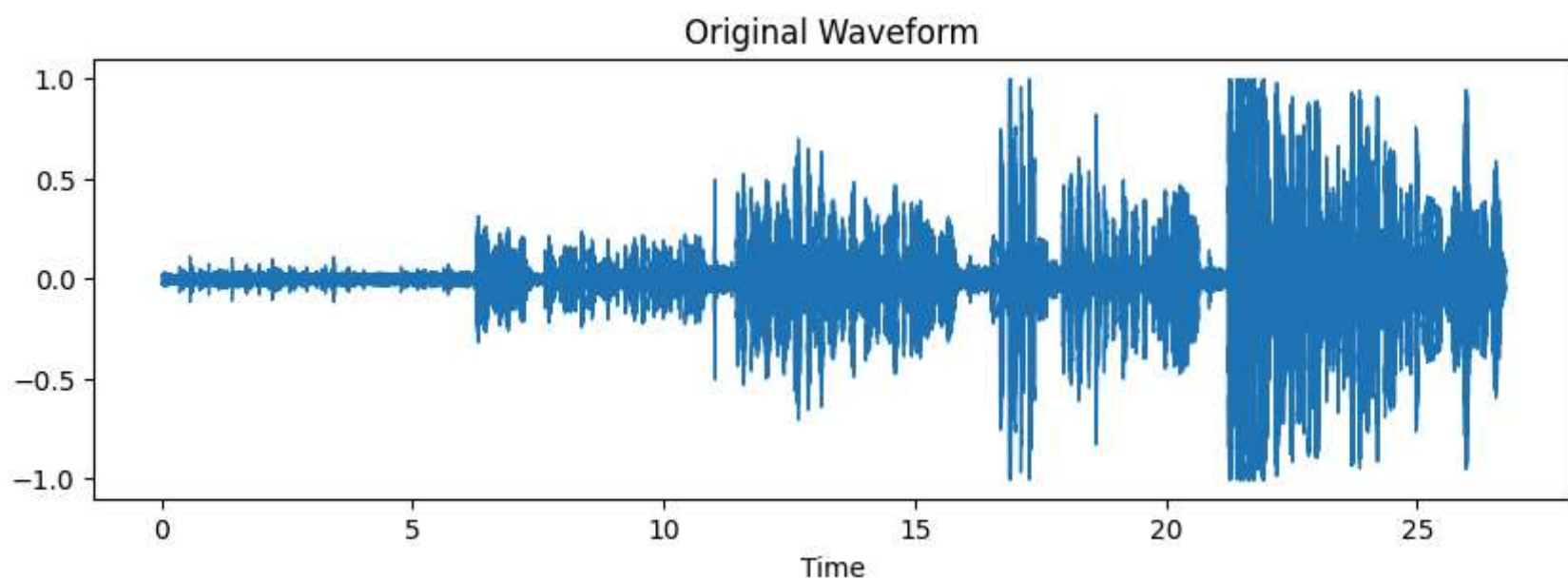
y, sr = librosa.load("audio1.wav", sr=None)

display(Audio(y, rate=sr))

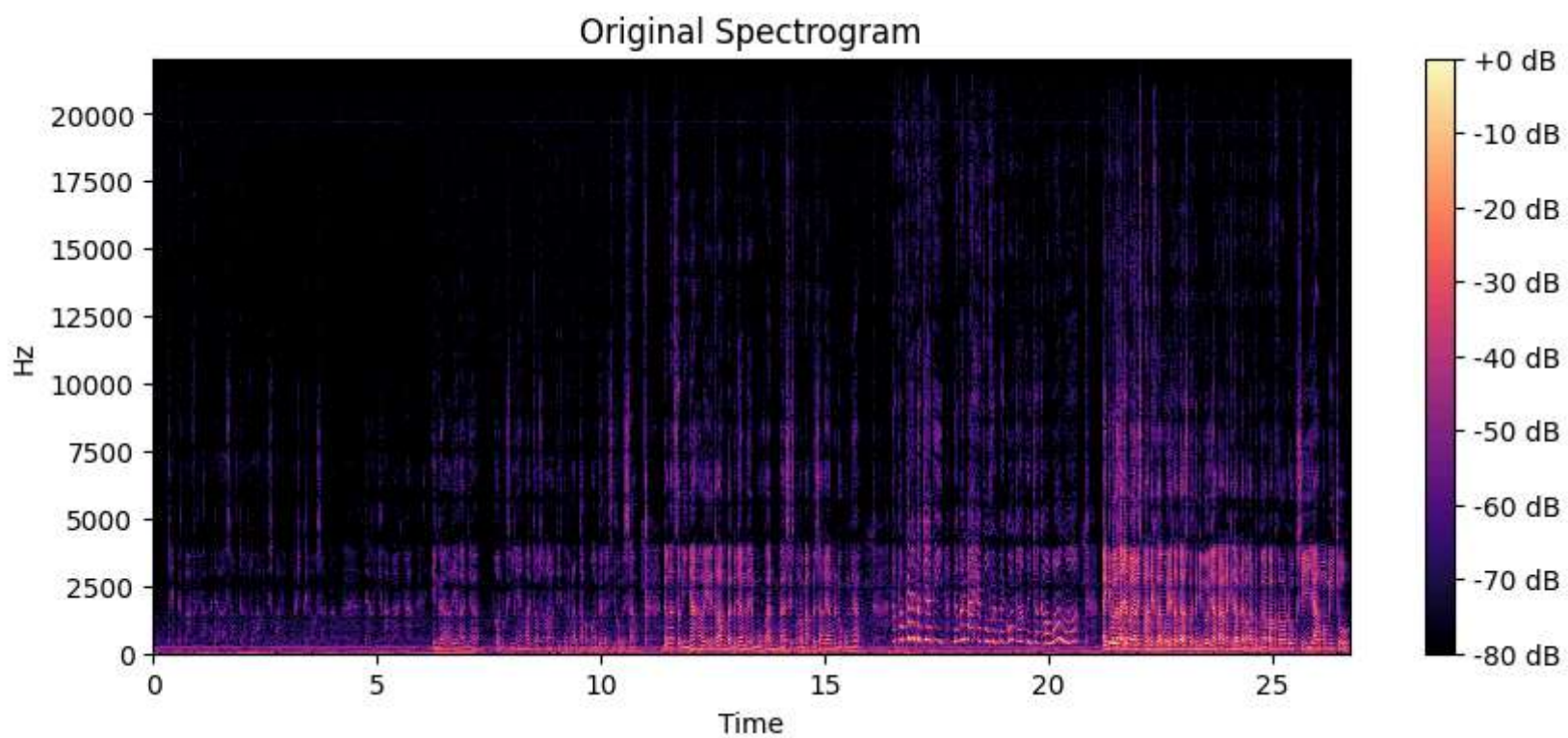
plt.figure(figsize=(10,3))
librosa.display.waveshow(y, sr=sr)
plt.title("Original Waveform")
plt.show()

plt.figure(figsize=(10,4))
librosa.display.specshow(librosa.amplitude_to_db(librosa.stft(y), ref=np.max),
                        sr=sr, x_axis='time', y_axis='hz')
plt.title("Original Spectrogram")
plt.colorbar(format='%+2.0f dB')
plt.show()
```

▶ 0:00 / 0:26 — 🔊 ⋮



```
C:\Users\Useradmin\AppData\Local\Temp\ipykernel_5540\2639073886.py:20: UserWarning: amplitude_to_db was called on complex input
so phase information will be discarded. To suppress this warning, call amplitude_to_db(np.abs(S)) instead.
librosa.display.specshow(librosa.amplitude_to_db(librosa.stft(y), ref=np.max),
```



Pada waveform terlihat jika setiao ≥ 5 detik gelombangnya semakin membesar, karena ya sesuai sama diminta tugas. Untuk spectrogram nya bisa diliat jika frequency yang dominan adalah sekitar 5000hz yang merupakan frekuensi suara manusia (vokal).

```
In [9]: # ===== Resampling =====
target_sr = 96000
if sr != target_sr:
    y_resampled = librosa.resample(y, orig_sr=sr, target_sr=target_sr)
    print(f"Audio resampled from {sr} Hz to {target_sr} Hz")
    sr = target_sr
else:
    y_resampled = y

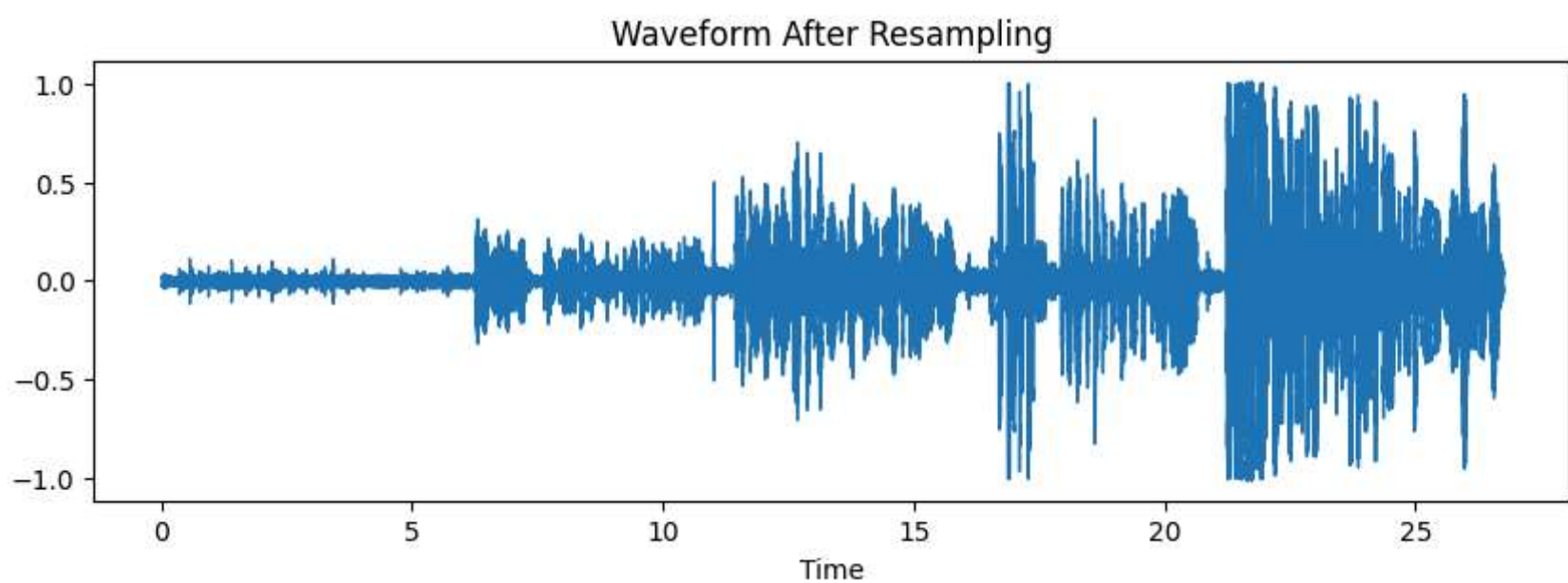
print("Resampled audio (Sample rate: {}) Hz:".format(target_sr))
display(Audio(y_resampled, rate=target_sr))

plt.figure(figsize=(10,3))
librosa.display.waveshow(y_resampled, sr=sr)
plt.title("Waveform After Resampling")
plt.show()

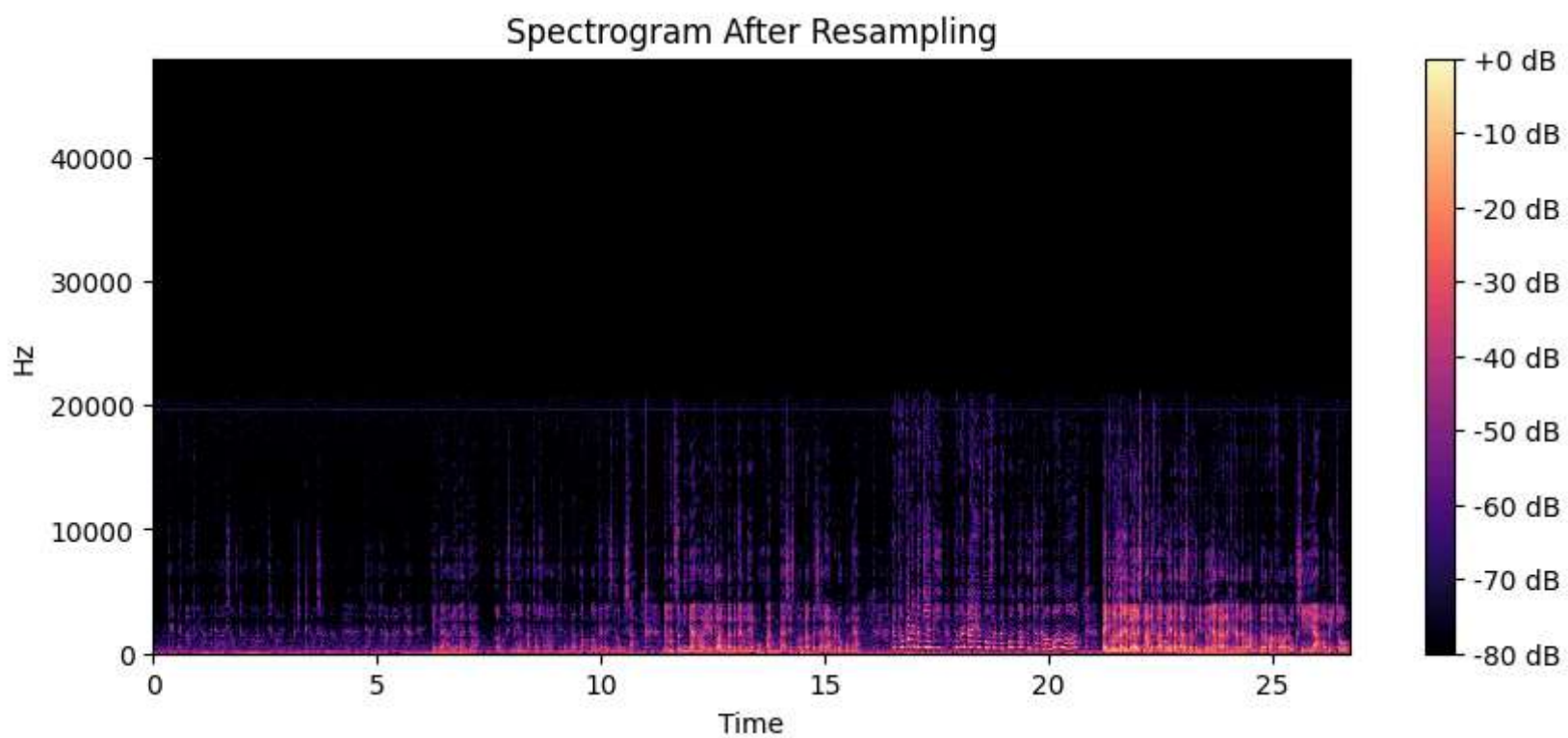
plt.figure(figsize=(10,4))
librosa.display.specshow(librosa.amplitude_to_db(librosa.stft(y_resampled), ref=np.max),
                        sr=sr, x_axis='time', y_axis='hz')
plt.title("Spectrogram After Resampling")
plt.colorbar(format='%+2.0f dB')
plt.show()
```

Audio resampled from 44100 Hz to 96000 Hz
Resampled audio (Sample rate: 96000 Hz):

▶ 0:00 / 0:26 🔊 ⋮



C:\Users\Useradmin\AppData\Local\Temp\ipykernel_5540\2376953437.py:19: UserWarning: amplitude_to_db was called on complex input so phase information will be discarded. To suppress this warning, call amplitude_to_db(np.abs(S)) instead.
librosa.display.specshow(librosa.amplitude_to_db(librosa.stft(y_resampled), ref=np.max),



Resampling yang saya lakukan membuat audio yang dari 44.1 khz menjadi 96 khz, kalau dari waveform gak keliatan, tapi kalau dari spectrogram bisa terlihat frequency nya menjadi 48 khz. Tidak ada perbedaan kualitas maupun perbedaan durasi, karena resampling tidak membuat durasinya bertambah.

Soal 2: Noise Reduction dengan Filtering • Rekam suara Anda berbicara di sekitar objek yang berisik (seperti kipas angin, AC, atau mesin). – Rekaman tersebut harus berdurasi kurang lebih 10 detik. – Rekam dalam format WAV (atau konversikan ke WAV sebelum dimuat ke notebook). • Gunakan filter equalisasi (high-pass, low-pass, dan band-pass) untuk menghilangkan noise pada rekaman tersebut. • Lakukan eksperimen dengan berbagai nilai frekuensi cutoff (misalnya 500 Hz, 1000 Hz, 2000 Hz). • Visualisasikan hasil dari tiap filter dan bandingkan spektrogramnya.

```
In [10]: y, sr = librosa.load("audio2.wav", sr=None)

display(Audio(y, rate=sr))

def butter_filter(data, cutoff, sr, btype, order=5):
    nyq = 0.5 * sr
    norm = cutoff/nyq if isinstance(cutoff,(int,float)) else [c/nyq for c in cutoff]
    b, a = butter(order, norm, btype=btype)
    return filtfilt(b, a, data)

# High-pass 1000 Hz
high_800 = butter_filter(y, 800, sr, 'high')
if not np.isfinite(high_800).all():
    high_800 = np.nan_to_num(high_800)

print("\nHigh-pass 800 Hz filtered audio:")
display(Audio(high_800, rate=sr))
D = librosa.amplitude_to_db(np.abs(librosa.stft(high_800)), ref=np.max)
plt.figure(figsize=(10, 4))
librosa.display.specshow(D, sr=sr, x_axis='time', y_axis='hz', cmap='magma')
plt.colorbar(format='%+2.0f dB')
plt.title("Spectrogram - High-pass 800 Hz")
plt.tight_layout()
plt.show()

# Low-pass 7500 Hz
low_5000 = butter_filter(y, 5000, sr, 'low')
if not np.isfinite(low_5000).all():
    low_1000 = np.nan_to_num(low_5000)

print("\nLow-pass 6000 Hz filtered audio:")
display(Audio(low_5000, rate=sr))
D = librosa.amplitude_to_db(np.abs(librosa.stft(low_5000)), ref=np.max)
plt.figure(figsize=(10, 4))
librosa.display.specshow(D, sr=sr, x_axis='time', y_axis='hz', cmap='magma')
plt.colorbar(format='%+2.0f dB')
plt.title("Spectrogram - Low-pass 6000 Hz")
plt.tight_layout()
plt.show()

# Band-pass 1000-7500 Hz
band_800_6000 = butter_filter(y, [800, 6000], sr, 'band')
if not np.isfinite(band_800_6000).all():
    band_800_6000 = np.nan_to_num(band_800_6000)

print("\nBand-pass 800-6000 Hz filtered audio:")
display(Audio(band_800_6000, rate=sr))
D = librosa.amplitude_to_db(np.abs(librosa.stft(band_800_6000)), ref=np.max)
```

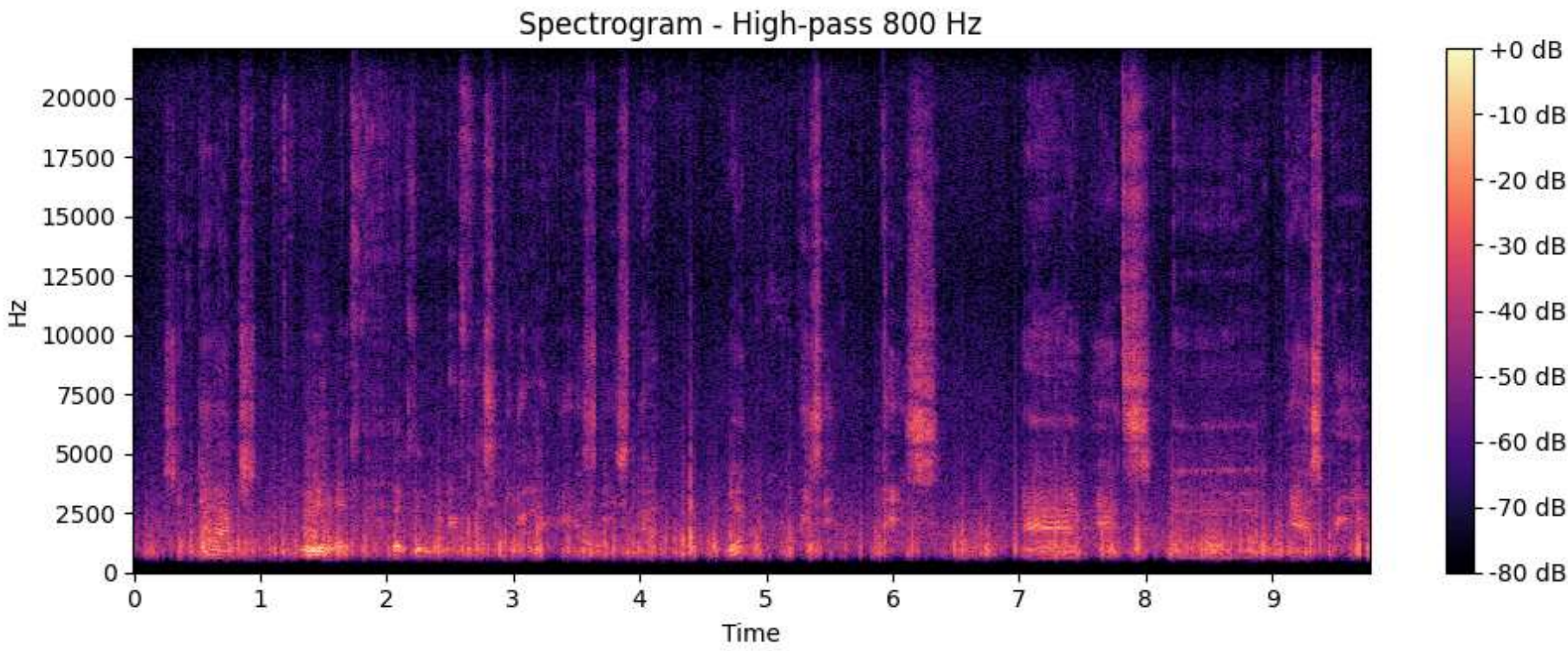


```
plt.figure(figsize=(10, 4))
librosa.display.specshow(D, sr=sr, x_axis='time', y_axis='hz', cmap='magma')
plt.colorbar(format='%+2.0f dB')
plt.title("Spectrogram - Band-pass 800-6000 Hz")
plt.tight_layout()
plt.show()
```

▶ 0:00 / 0:09 — 🔊 ⋮

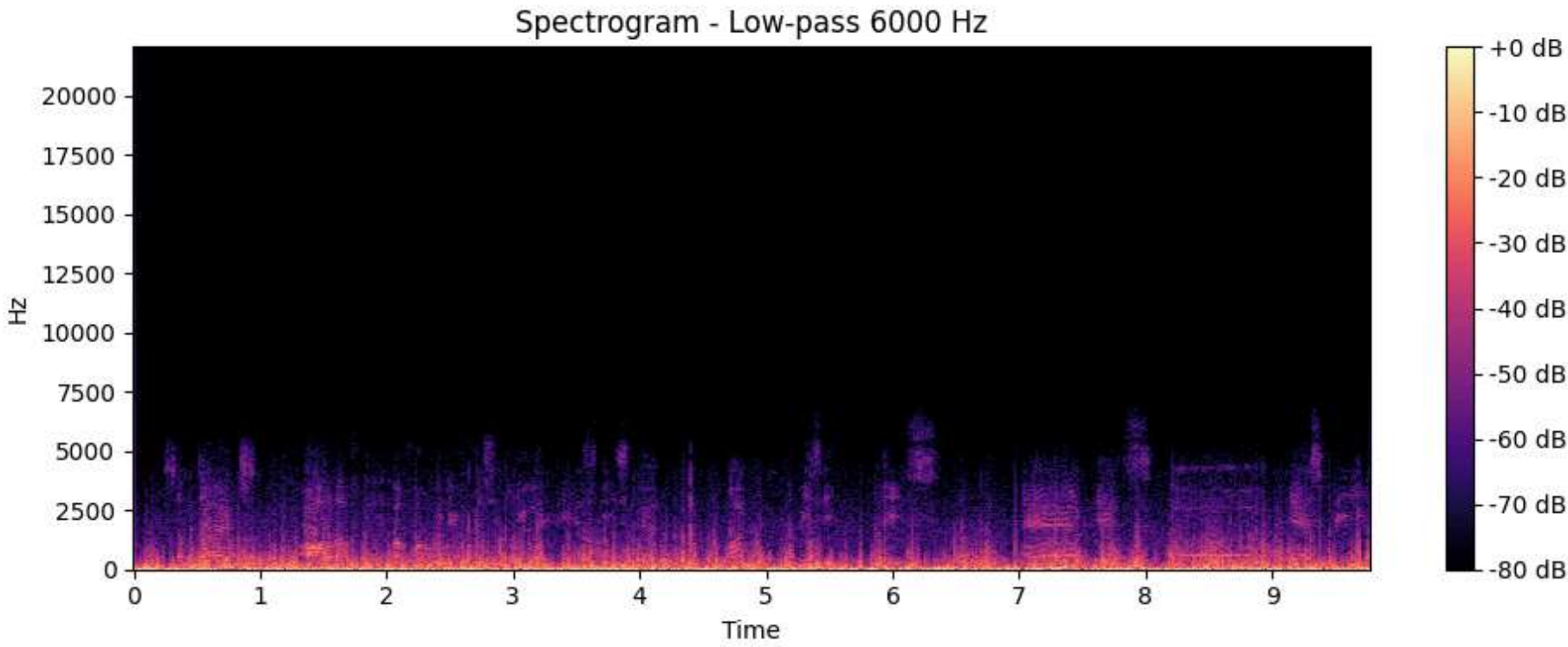
High-pass 800 Hz filtered audio:

▶ 0:00 / 0:09 — 🔊 ⋮



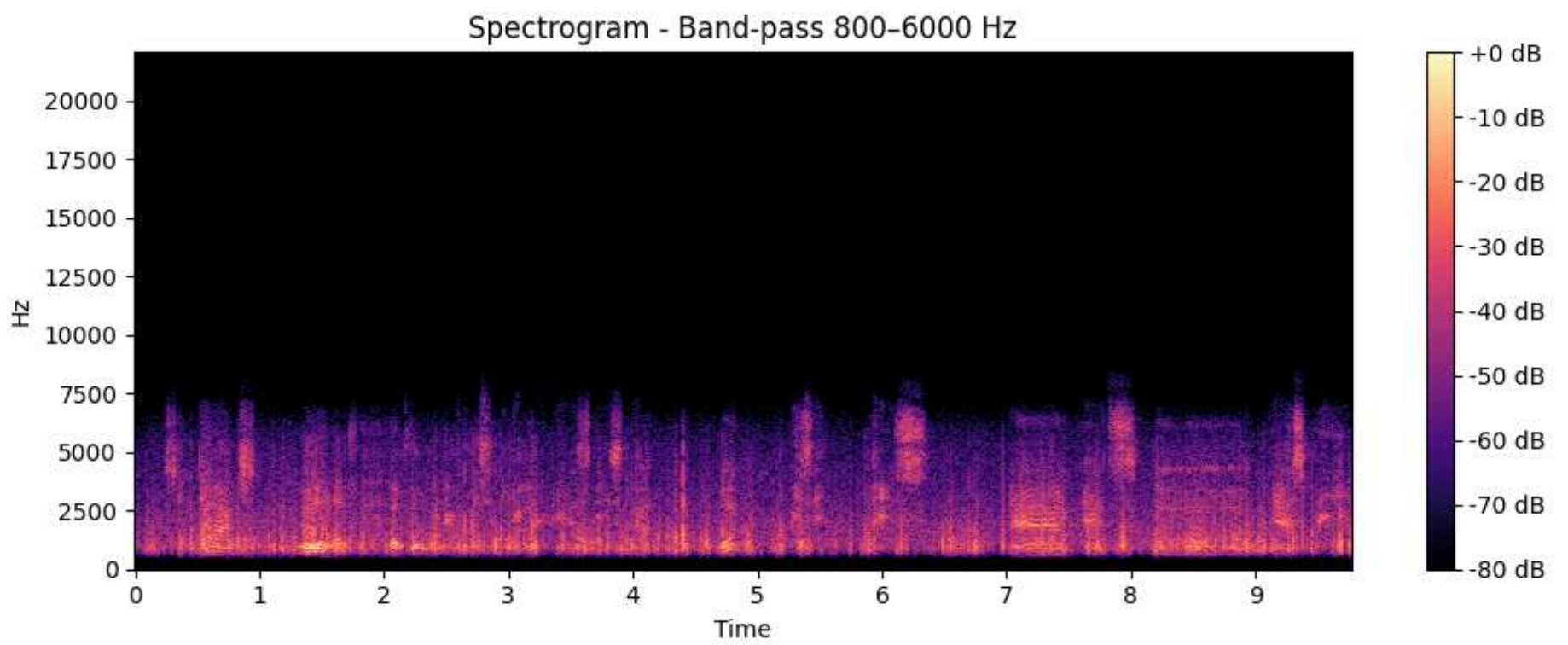
Low-pass 6000 Hz filtered audio:

▶ 0:00 / 0:09 — 🔊 ⋮



Band-pass 800-6000 Hz filtered audio:

▶ 0:00 / 0:09 — 🔊 ⋮



Jelaskan:

– Jenis noise yang muncul pada rekaman Anda – Filter mana yang paling efektif untuk mengurangi noise tersebut – Nilai cutoff yang memberikan hasil terbaik – Bagaimana kualitas suara (kejelasan ucapan) setelah proses filtering

Jenis noise yang ada merupakan dari suara kipas angin yang continue, bukan seperti batuk atau ketukan di sebuah lagu. Filter yang paling efektif adalah filter Band-Pass, karena filter ini memotong kedua sisi ujung spektrum. Nilai cutoff yang baik adalah 800 hz dan 6000 hz, karena cut di 800 hz membuat suara "kelelep" menghilang, dan pada 6000 hz suara kipasnya menjadi hilang (bersih). Kualitas suara dan kejelasan ucapan sangat meningkat.

Soal 3: Pitch Shifting dan Audio Manipulation • Lakukan pitch shifting pada rekaman suara Soal 1 untuk membuat suara terdengar seperti chipmunk (dengan mengubah pitch ke atas). • Visualisasikan waveform dan spektrogram sebelum dan sesudah pitch shifting. • Gunakan dua buah pitch tinggi, misalnya pitch +7 dan pitch +12. • Gabungkan kedua rekaman yang telah di-pitch shift ke dalam satu file audio. (Gunakan ChatGPT / AI untuk membantu Anda dalam proses ini)

```
In [11]: import librosa
import librosa.display
import numpy as np
import matplotlib.pyplot as plt
import soundfile as sf
from IPython.display import Audio

# LOAD AUDIO
y, sr = librosa.load("audio1.wav", sr=None)

# PITCH SHIFT +7 SEMITONES
y_pitch7 = librosa.effects.pitch_shift(y, sr=sr, n_steps=7)

# PITCH SHIFT +12 SEMITONES (one octave)
y_pitch12 = librosa.effects.pitch_shift(y, sr=sr, n_steps=12)

# CONCATENATE: pitch +7 first, then pitch +12
y_concat = np.concatenate([y_pitch7, y_pitch12])

# SAVE THE RESULT
sf.write('audio3.wav', y_concat, sr)

# OPTIONAL: Listen to the result (in Jupyter)
print("Concatenated Audio (Pitch +7 then +12):")
display(Audio(y_concat, rate=sr))

# PLAY AUDIO
print("Original audio:")
display(Audio(y, rate=sr))

print("Pitch shifted +7 semitones:")
display(Audio(y_pitch7, rate=sr))

print("Pitch shifted +12 semitones (1 octave):")
display(Audio(y_pitch12, rate=sr))

# Visualization before pitching
plt.figure(figsize=(12,8))
plt.subplot(2,1,1)
librosa.display.waveshow(y, sr=sr)
plt.title("Waveform Original (audio3.wav)")
plt.subplot(2,1,2)
D0 = librosa.amplitude_to_db(np.abs(librosa.stft(y)), ref=np.max)
librosa.display.specshow(D0, sr=sr, x_axis='time', y_axis='hz')
```

```
plt.title("Spectrogram Original")
plt.colorbar(format='%+2.0f dB')
plt.tight_layout()
plt.show()

# Visualization after pitch +7
plt.figure(figsize=(12,8))
plt.subplot(2,1,1)
librosa.display.waveshow(y_pitch7, sr=sr)
plt.title("Waveform After Pitching +7")
plt.subplot(2,1,2)
D7 = librosa.amplitude_to_db(np.abs(librosa.stft(y_pitch7)), ref=np.max)
librosa.display.specshow(D7, sr=sr, x_axis='time', y_axis='hz')
plt.title("Pitch +7 semitones - Spectrogram")
plt.colorbar(format='%+2.0f dB')
plt.tight_layout()
plt.show()

# Visualize after pitch +12
plt.figure(figsize=(12,8))
plt.subplot(2,1,1)
librosa.display.waveshow(y_pitch12, sr=sr)
plt.title("Waveform After Pitching +12")
plt.subplot(2,1,2)
D12 = librosa.amplitude_to_db(np.abs(librosa.stft(y_pitch12)), ref=np.max)
librosa.display.specshow(D12, sr=sr, x_axis='time', y_axis='hz')
plt.title("Pitch +12 semitones - Spectrogram")
plt.colorbar(format='%+2.0f dB')

plt.tight_layout()
plt.show()
```

Concatenated Audio (Pitch +7 then +12):

▶

0:00 / 0:53

⋮

Original audio:

▶

0:00 / 0:26

⋮

Pitch shifted +7 semitones:

▶

0:00 / 0:26

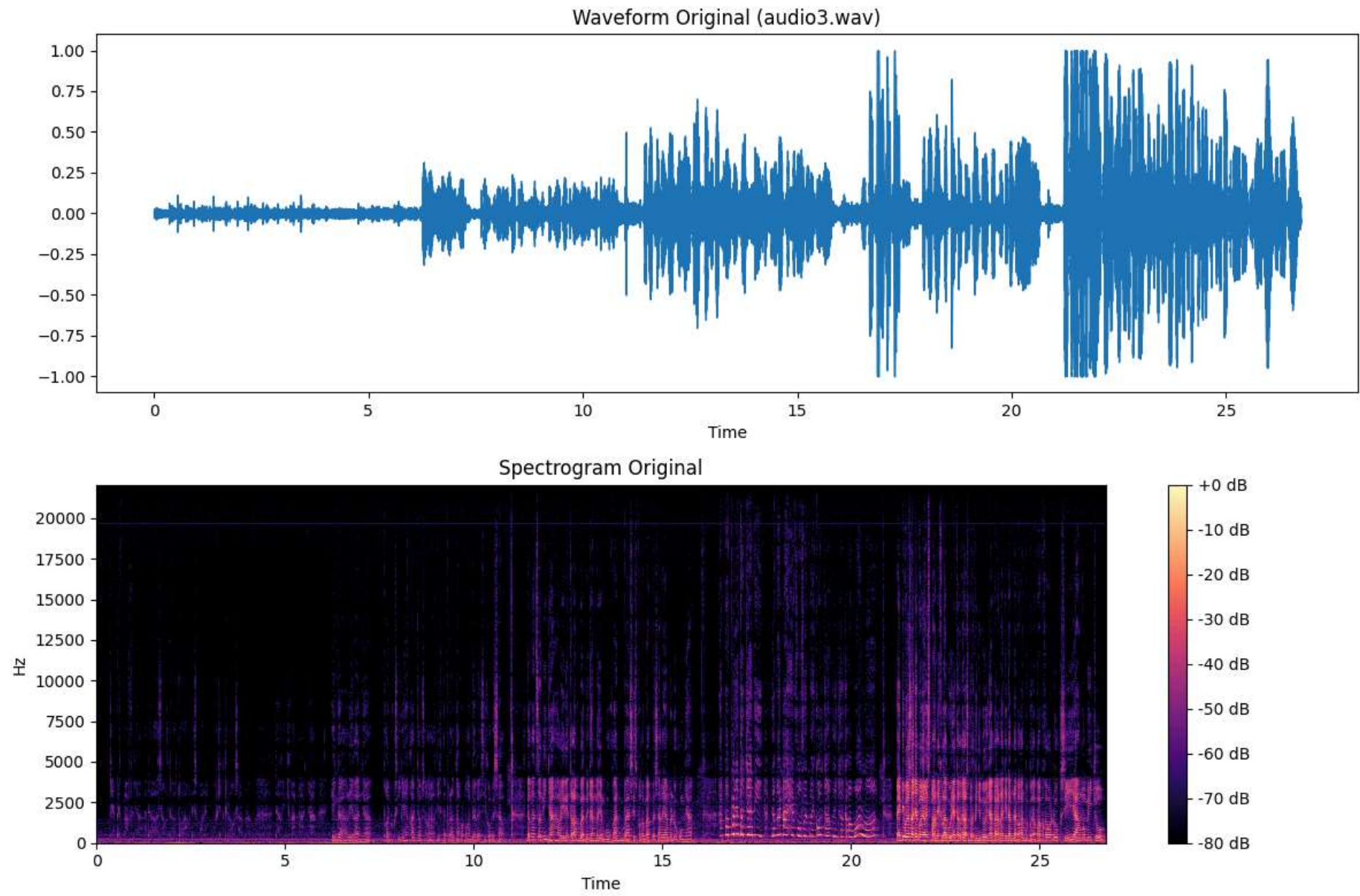
⋮

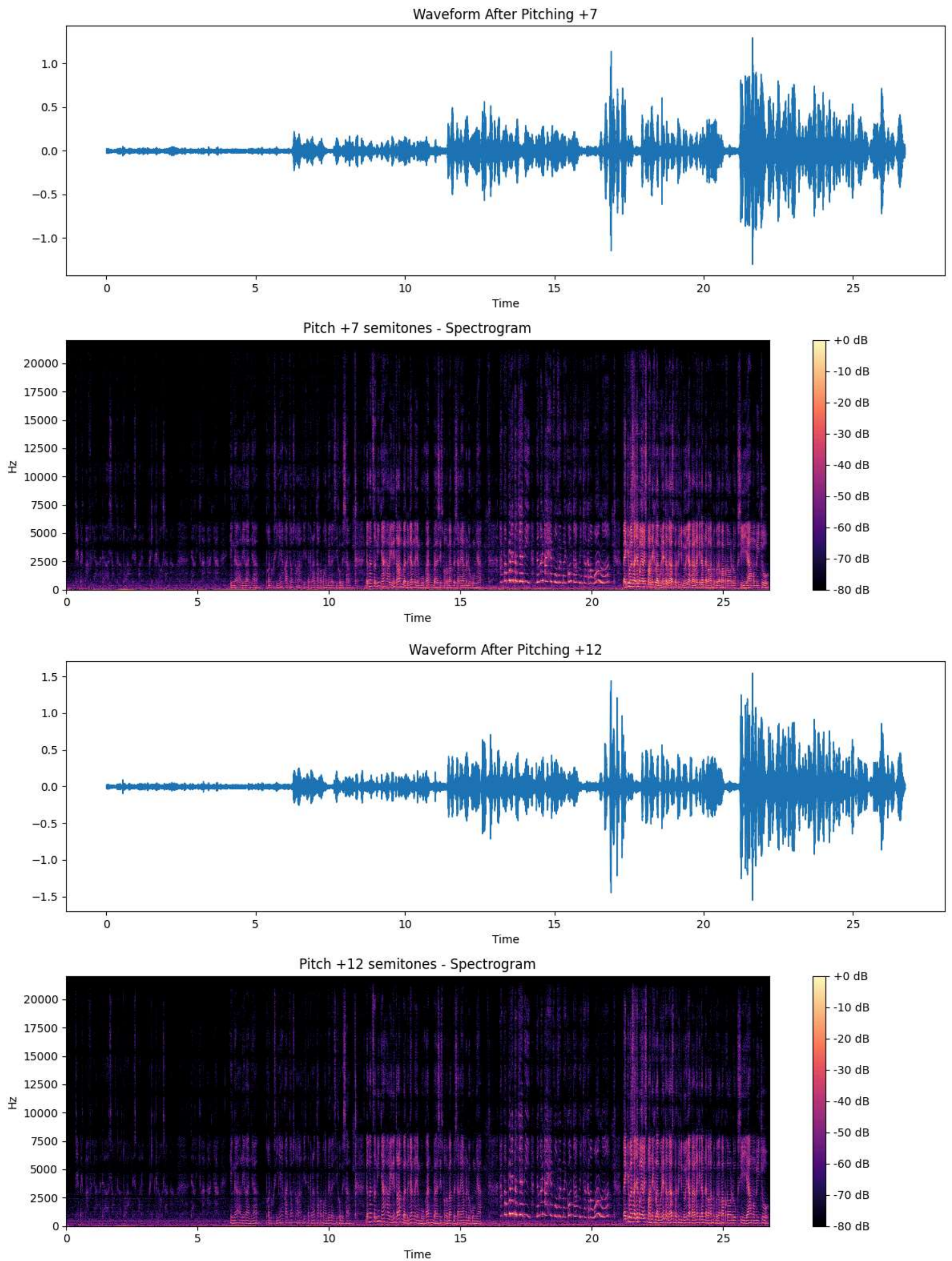
Pitch shifted +12 semitones (1 octave):

▶

0:00 / 0:26

⋮





Jelaskan proses pitch shifting yang Anda lakukan, termasuk:

– Parameter yang digunakan – Perbedaan dalam representasi visual antara suara asli dan suara yang telah dimodifikasi. – Bagaimana perubahan pitch memengaruhi kualitas dan kejelasan suara

Parameter yang digunakan ada audi1.wav, kemudian sr nya tidak ada berubah, pich yang digunakan adaalah +7 dan +12. Tidak ada perubahan pada waveform, tetapi pada spectrogram dapat dilihat jika semua pola suara spektogram + 12 dan +7 keseluruhan berada pada posisi vertikal yang lebih tinggi, ini berarti frekuensi nada telah digeser keatas .Perubahan pitch tidak mempengaruhi kejelasan suara, hanya saja suaranya menjadi lebih tinggi.

Soal 4: Audio Processing Chain • Lakukan processing pada rekaman yang sudah di-pitch shift pada Soal 3 dengan tahapan: – Equalizer – Gain/fade – Normalization – Compression – Noise Gate – Silence trimming

```
In [12]: from scipy.signal import butter, lfilter

# LOAD audio3.wav
y, sr = librosa.load("audio3.wav", sr=None)

# ==== EQUALIZER: Parametric (example: cut <200Hz, boost 1kHz-5kHz) ====
def eq_band(y, sr, kind='band', freq=1000, gain_db=6.0, Q=1):
    # Butterworth filter as simple EQ
    nyq = 0.5 * sr
    norm_freq = freq / nyq
    if kind == 'band':
        b, a = butter(2, [norm_freq/Q, norm_freq*Q], btype='band')
    elif kind == 'low':
        b, a = butter(2, norm_freq, btype='low')
    elif kind == 'high':
        b, a = butter(2, norm_freq, btype='high')
    else:
        return y
    filtered = lfilter(b, a, y)
    gain = 10**((gain_db/20))
    return y + gain * filtered

# Example: cut bass, boost mid
y_eq = eq_band(y, sr, kind='low', freq=200, gain_db=-6)
y_eq = eq_band(y_eq, sr, kind='band', freq=3000, gain_db=5, Q=2)

# ==== GAIN/FADE ====
fade_in = np.linspace(0, 1, int(0.05*sr))
fade_out = np.linspace(1, 0, int(0.05*sr))
y_eq[:fade_in.size] *= fade_in      # Fade-in 50 ms
y_eq[-fade_out.size:] *= fade_out   # Fade-out 50 ms

# ==== NORMALIZATION (peak normalization) ====
peak = np.abs(y_eq).max()
y_norm = y_eq / (peak + 1e-8)

# ==== COMPRESSION ====
def compress(audio, threshold=0.5, ratio=4):
    # Simple static compressor
    audio_out = np.copy(audio)
    over_thr = np.abs(audio_out) > threshold
    audio_out[over_thr] = np.sign(audio_out[over_thr]) * (
        threshold + (np.abs(audio_out[over_thr]) - threshold) / ratio)
    return audio_out
y_comp = compress(y_norm, threshold=0.3, ratio=3)

# ==== NOISE GATE ====
def noise_gate(audio, gate_thresh=0.02):
    audio_out = np.copy(audio)
    audio_out[np.abs(audio_out) < gate_thresh] = 0
    return audio_out
y_gate = noise_gate(y_comp, gate_thresh=0.03)

# ==== TRIM SILENCE ====
y_trimmed, _ = librosa.effects.trim(y_gate, top_db=30)
```

- Atur nilai target loudness ke -16 LUFS.
- Visualisasikan waveform dan spektrogram sebelum dan sesudah proses normalisasi.

```
In [13]: # ==== LUFS NORMALIZATION (PERCEIVED LOUDNESS) ====meter = pyln.Meter(sr)
meter = pyln.Meter(sr)
loudness = meter.integrated_loudness(y_trimmed)
target_lufs = -16.0
y_lufs = pyln.normalize.loudness(y_trimmed, loudness, target_lufs)

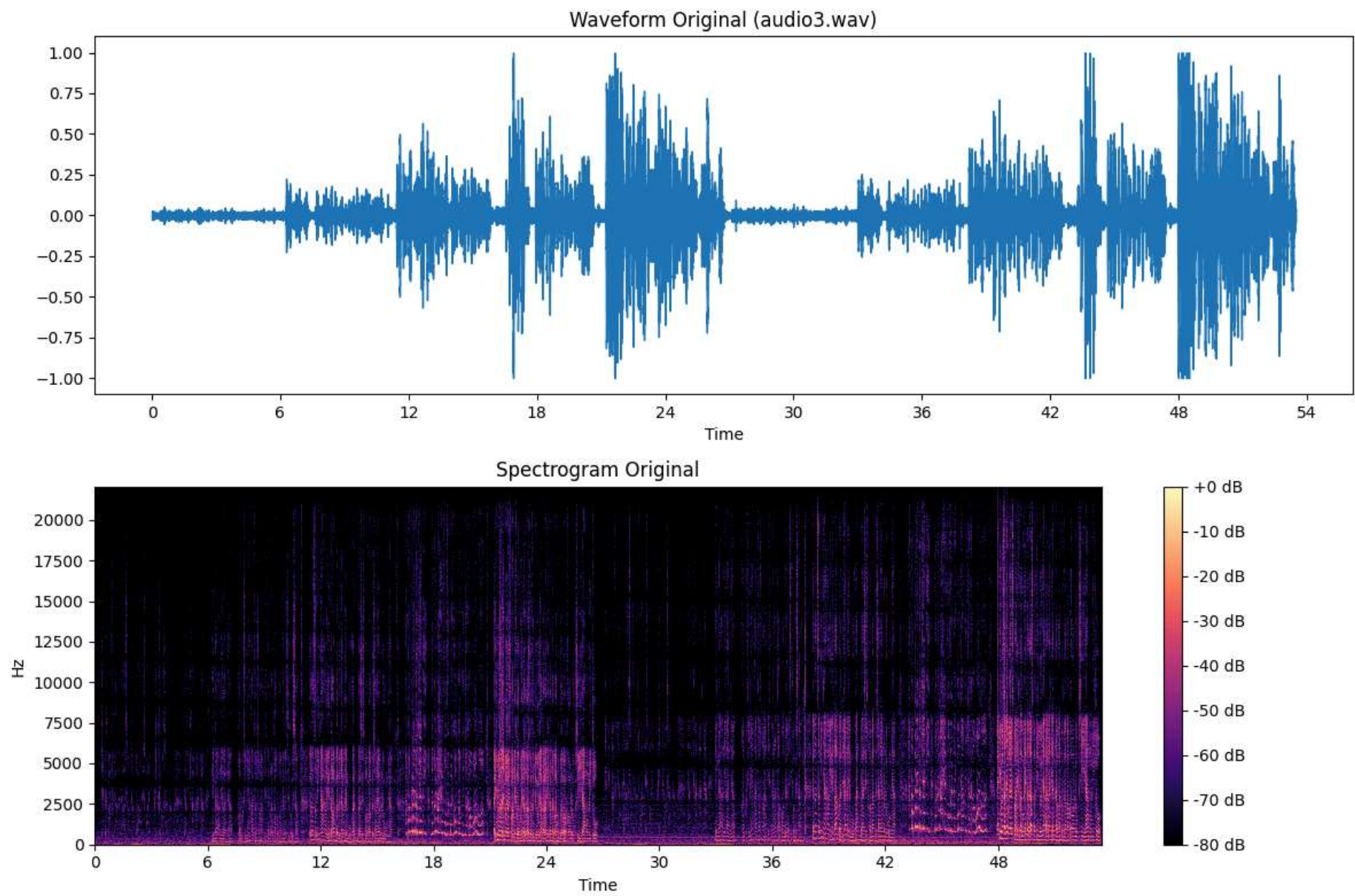
# ==== SAVE and Visualize ====
sf.write('audio4.wav', y_lufs, sr)

# Visualization before normalisasi
display(Audio(y, rate=sr))
plt.figure(figsize=(12,8))
plt.subplot(2,1,1)
librosa.display.waveshow(y, sr=sr)
plt.title("Waveform Original (audio3.wav)")
plt.subplot(2,1,2)
D0 = librosa.amplitude_to_db(np.abs(librosa.stft(y)), ref=np.max)
librosa.display.specshow(D0, sr=sr, x_axis='time', y_axis='hz')
plt.title("Spectrogram Original")
plt.colorbar(format='%+2.0f dB')
plt.tight_layout()
plt.show()
```

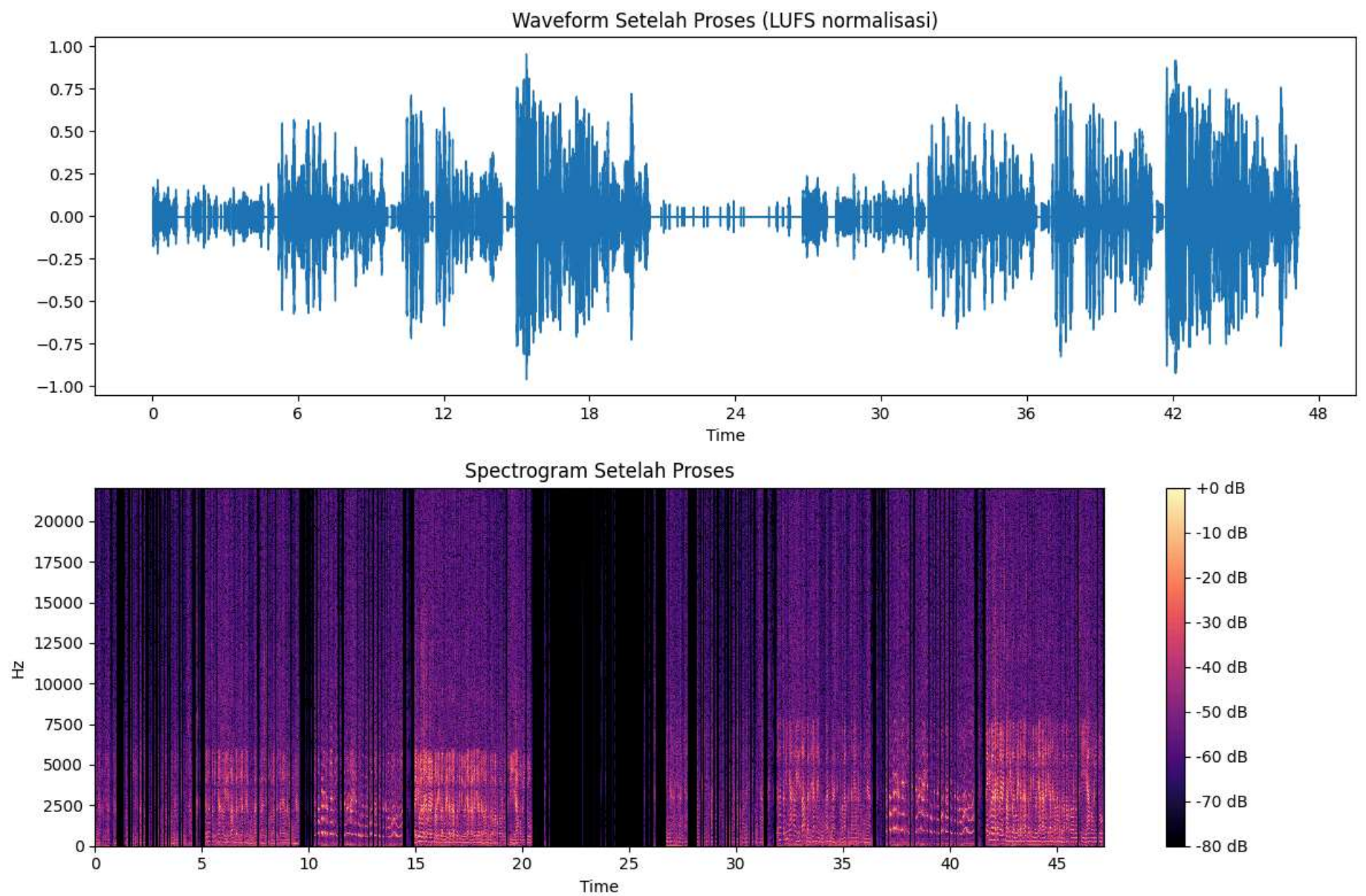


```
# Visualization sesudah normalisasi
display(Audio(y_lufs, rate=sr))
plt.figure(figsize=(12,8))
plt.subplot(2,1,1)
librosa.display.waveshow(y_lufs, sr=sr)
plt.title("Waveform Setelah Proses (LUFS normalisasi)")
plt.subplot(2,1,2)
D1 = librosa.amplitude_to_db(np.abs(librosa.stft(y_lufs)), ref=np.max)
librosa.display.specshow(D1, sr=sr, x_axis='time', y_axis='hz')
plt.title("Spectrogram Setelah Proses")
plt.colorbar(format='%+2.0f dB')
plt.tight_layout()
plt.show()
```

▶ 0:00 / 0:53 ————— 🔊 ⋮



▶ 0:00 / 0:47 ————— 🔊 ⋮



Jelaskan:

- Perubahan dinamika suara yang terjadi audio asli telah diproses untuk mempersempit rentang dinamisnya (melalui kompresi), membersihkan bagian-bagian yang hening (melalui noise gate), dan menggunakan equalizer untuk mengurangi suara yang mendem tersebut.
- Perbedaan antara normalisasi peak dan normalisasi LUFS . Peak Normalization merupakan sebuah proses editing audio dengan mengatur seluruh sinyal audio mencapai puncak tertinggi (0 dBFS). Sedangkan normalisasi LUFS adalah mengatur average perceive loudness, hal ini agar keseluruhan suara terdengar lebih konsisten di telinga: suara pelan menjadi lebih terdengar, suara keras diturunkan, sehingga transisi antar bagian lebih mulus dan nyaman.
- Bagaimana kualitas suara berubah setelah proses normalisasi dan loudness optimization Kualitas suara yang dihasilkan menjadi lebih stabil dan konsisten karena beberapa proses utama yang diterapkan. Perubahan ini secara kolektif mempersempit rentang dinamis, membersihkan audio, dan menyeimbangkan frekuensi agar lebih nyaman didengar.
- Kelebihan dan kekurangan dari pengoptimalan loudness dalam konteks rekaman suara: Kelebihannya volume antar track menjadi konsisten, dan juga kekurangannya karena volume antar track menjadi konsisten, jadi agak cukup mengganggu bila kita ingin mendengar lagu ber genre ukbass, experimental, dan trance.

Soal 5: Music Analysis dan Remix Pilih 2 buah (potongan) lagu yang memiliki vokal (penyanyi) dan berdurasi sekitar 1 menit: – Lagu 1: Nuansa sedih, lambat – Lagu 2: Nuansa ceria, cepat • Konversikan ke format WAV sebelum dimuat ke notebook. • Lakukan deteksi tempo (BPM) dan estimasi kunci (key) dari masing-masing lagu dan berikan analisis singkat. • Lakukan remix terhadap kedua lagu: – Time Stretch: Samakan tempo kedua lagu – Pitch Shift: Samakan kunci (key) kedua lagu – Crossfading: Gabungkan kedua lagu dengan efek crossfading – Filter Tambahan: Tambahkan filter kreatif sesuai keinginan (opsional)

```
In [14]: # ===== PARAMETERS =====
song1_path = 'Sad.wav'      # Nuansa sedih, lambat
song2_path = 'Happy.wav'   # Nuansa ceria, cepat

# ===== LOAD AUDIO FILES =====
y1, sr1 = librosa.load(song1_path, sr=None)
y2, sr2 = librosa.load(song2_path, sr=None)

def analyze_audio(y, sr, name):
    tempo, _ = librosa.beat.beat_track(y=y, sr=sr)
    tempo = float(tempo)
    chroma = librosa.feature.chroma_cqt(y=y, sr=sr)
    chroma_avg = chroma.mean(axis=1)
    key_index = np.argmax(chroma_avg)
    key_names = ['C', 'C#', 'D', 'D#', 'E', 'F', 'F#', 'G', 'G#', 'A', 'A#', 'B']
    key = key_names[key_index]
    print(f"{name}: BPM={tempo:.1f}, Key={key}")
    return tempo, key

# ANALYZE FIRST!
bpm1, key1 = analyze_audio(y1, sr1, "Lagu 1 (Sad)")
```



```

bpm2, key2 = analyze_audio(y2, sr2, "Lagu 2 (Happy)")

# ===== TIME-STRETCH: MATCH TEMPO =====
def time_stretch(y, orig_bpm, target_bpm):
    rate = float(orig_bpm) / float(target_bpm)
    return librosa.effects.time_stretch(y, rate=rate)

y1_stretched = y1 # Lagu 1 tetap
y2_stretched = time_stretch(y2, float(bpm2), float(bpm1))

# ===== PITCH-SHIFT: MATCH KEY =====
def semitone_shift(orig_key, target_key):
    key_names = ['C', 'C#', 'D', 'D#', 'E', 'F', 'F#', 'G', 'G#', 'A', 'A#', 'B']
    shift = key_names.index(target_key) - key_names.index(orig_key)
    if shift > 6:
        shift -= 12
    elif shift < -6:
        shift += 12
    return shift

shift_amt = semitone_shift(key2, key1)
y2_shifted = librosa.effects.pitch_shift(y=y2_stretched, sr=sr2, n_steps=shift_amt)
y1_shifted = y1_stretched # No shift needed

# ===== CROSSFADING =====
def crossfade(y1, y2, sr, duration_sec=4.0):
    fade_len = int(sr * duration_sec)
    fade_out = np.linspace(1, 0, fade_len)
    fade_in = np.linspace(0, 1, fade_len)
    y1_solo = y1[:-fade_len]
    y2_solo = y2[fade_len:]
    crossfade_section = y1[-fade_len:] * fade_out + y2[:fade_len] * fade_in
    return np.concatenate([y1_solo, crossfade_section, y2_solo])

remix = crossfade(y1_shifted, y2_shifted, sr1, duration_sec=4.0)

# ===== CREATIVE FILTERING =====
def creative_filter(y, sr):
    length = len(y)
    nyq = sr / 2
    p1 = length // 3
    p2 = 2 * length // 3
    out = y.copy()
    # High-pass on first third
    hp_b, hp_a = butter(4, 100/nyq, btype='high')
    out[:p1] = lfilter(hp_b, hp_a, out[:p1])

    # Band-pass on last third
    bp_b, bp_a = butter(3, [200/nyq, 8000/nyq], btype='band')
    out[p2:] = lfilter(bp_b, bp_a, out[p2:])
    return out

remix_filtered = creative_filter(remix, sr1)
remix_filtered /= np.max(np.abs(remix_filtered)) * 1.01 # Normalize peak just below 1

# ===== SAVE OUTPUT =====
sf.write('remix_sad_to_happy.wav', remix_filtered, sr1)
print("\nRemix saved as remix_sad_to_happy.wav")

# ===== VISUALIZATION =====
plt.figure(figsize=(15, 9))
plt.subplot(3,1,1)
librosa.display.waveshow(y1, sr=sr1, color='blue')
plt.title("Waveform Original Lagu 1 (Sad)")
plt.subplot(3,1,2)
librosa.display.waveshow(y2, sr=sr2, color='orange')
plt.title("Waveform Original Lagu 2 (Happy)")

plt.figure(figsize=(15, 9))
plt.subplot(3,1,1)
D1 = librosa.amplitude_to_db(np.abs(librosa.stft(y1)), ref=np.max)
librosa.display.specshow(D1, sr=sr1, x_axis='time', y_axis='hz')
plt.title("Spectrogram - Lagu 1 (Sad)")
plt.colorbar(format='%+2.0f dB')
plt.subplot(3,1,2)
D2 = librosa.amplitude_to_db(np.abs(librosa.stft(y2)), ref=np.max)
librosa.display.specshow(D2, sr=sr2, x_axis='time', y_axis='hz')
plt.title("Spectrogram - Lagu 2 (Happy)")
plt.colorbar(format='%+2.0f dB')

```

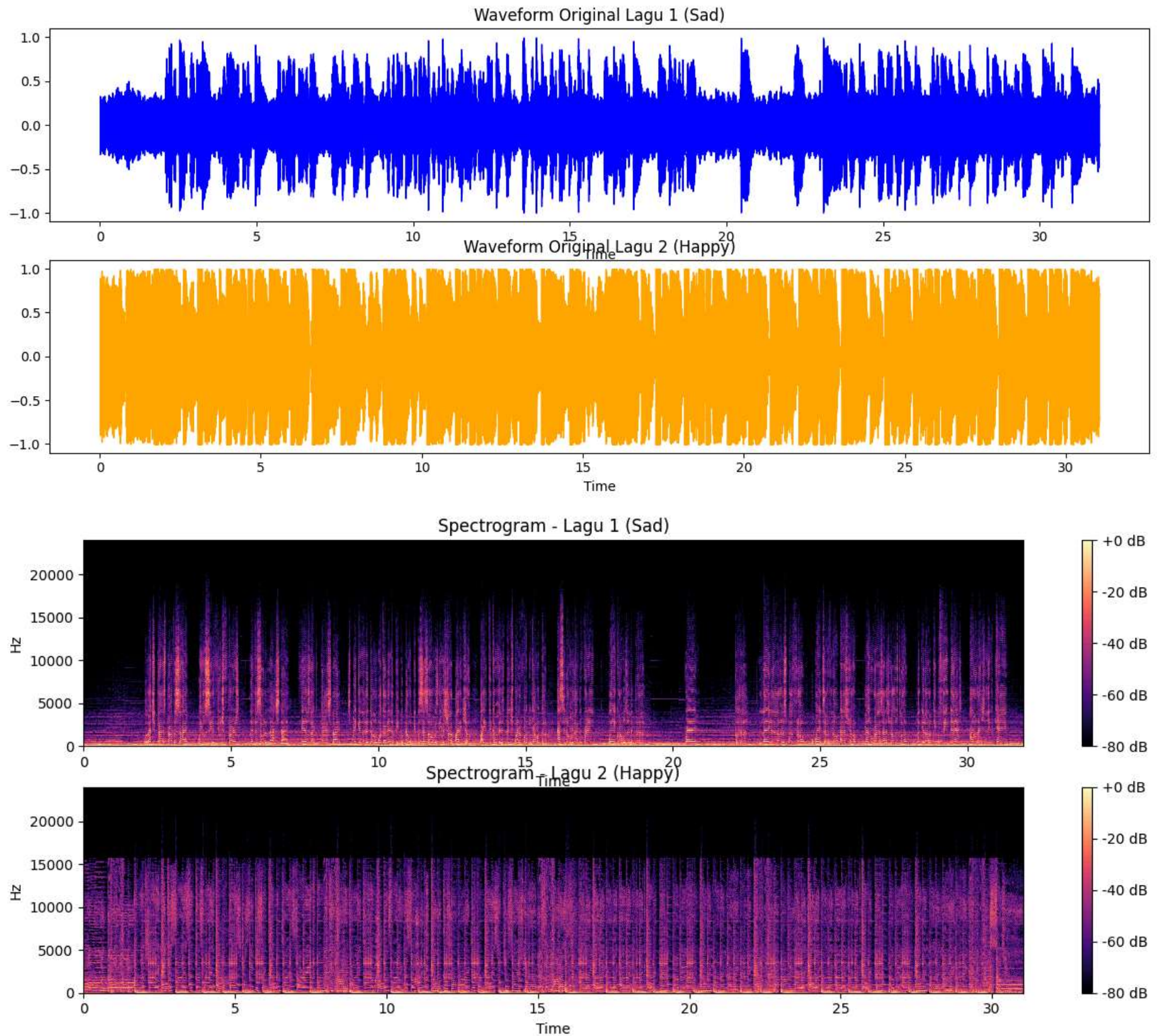
C:\Users\Useradmin\AppData\Local\Temp\ipykernel_5540\497829401.py:11: DeprecationWarning: Conversion of an array with ndim > 0 to a scalar is deprecated, and will error in future. Ensure you extract a single element from your array before performing this operation. (Deprecated NumPy 1.25.)

```
tempo = float(tempo)
```

Lagu 1 (Sad): BPM=100.4, Key=C
Lagu 2 (Happy): BPM=133.9, Key=F

Remix saved as remix_sad_to_happy.wav

Out[14]: <matplotlib.colorbar.Colorbar at 0x1dd117f8550>



```
In [15]: # ===== PARAMETERS =====
song1_path = 'Sad.wav'      # Nuansa sedih, Lambat
song2_path = 'Happy.wav'    # Nuansa ceria, cepat

# ===== LOAD AUDIO FILES =====
y1, sr1 = librosa.load(song1_path, sr=None)
y2, sr2 = librosa.load(song2_path, sr=None)

def analyze_audio(y, sr, name):
    tempo, _ = librosa.beat.beat_track(y=y, sr=sr)
    tempo = float(tempo)
    chroma = librosa.feature.chroma_cqt(y=y, sr=sr)
    chroma_avg = chroma.mean(axis=1)
    key_index = np.argmax(chroma_avg)
    key_names = ['C', 'C#', 'D', 'D#', 'E', 'F', 'F#', 'G', 'G#', 'A', 'A#', 'B']
    key = key_names[key_index]
    print(f"{name}: BPM={tempo:.1f}, Key={key}")
    return tempo, key

# ANALYZE FIRST!
bpm1, key1 = analyze_audio(y1, sr1, "Lagu 1 (Sad)")
bpm2, key2 = analyze_audio(y2, sr2, "Lagu 2 (Happy)")

# ===== TIME-STRETCH: MATCH TEMPO =====
def time_stretch(y, orig_bpm, target_bpm):
    rate = float(orig_bpm) / float(target_bpm)
    return librosa.effects.time_stretch(y, rate=rate)

y1_stretched = y1 # Lagu 1 tetap
y2_stretched = time_stretch(y2, float(bpm2), float(bpm1))
```



```

# ===== PITCH-SHIFT: MATCH KEY =====
def semitone_shift(orig_key, target_key):
    key_names = ['C', 'C#', 'D', 'D#', 'E', 'F', 'F#', 'G', 'G#', 'A', 'A#', 'B']
    shift = key_names.index(target_key) - key_names.index(orig_key)
    if shift > 6:
        shift -= 12
    elif shift < -6:
        shift += 12
    return shift

shift_amt = semitone_shift(key2, key1)
y2_shifted = librosa.effects.pitch_shift(y=y2_stretched, sr=sr2, n_steps=shift_amt)
y1_shifted = y1_stretched # No shift needed

# ===== CROSSFADING =====
def crossfade(y1, y2, sr, duration_sec=4.0):
    fade_len = int(sr * duration_sec)
    fade_out = np.linspace(1, 0, fade_len)
    fade_in = np.linspace(0, 1, fade_len)
    y1_solo = y1[:-fade_len]
    y2_solo = y2[fade_len:]
    crossfade_section = y1[-fade_len:] * fade_out + y2[:fade_len] * fade_in
    return np.concatenate([y1_solo, crossfade_section, y2_solo])

remix = crossfade(y1_shifted, y2_shifted, sr1, duration_sec=4.0)

# ===== CREATIVE FILTERING =====
def creative_filter(y, sr):
    length = len(y)
    nyq = sr / 2
    p1 = length // 3
    p2 = 2 * length // 3
    out = y.copy()
    # High-pass on first third
    hp_b, hp_a = butter(4, 100/nyq, btype='high')
    out[:p1] = lfilter(hp_b, hp_a, out[:p1])

    # Band-pass on last third
    bp_b, bp_a = butter(3, [200/nyq, 8000/nyq], btype='band')
    out[p2:] = lfilter(bp_b, bp_a, out[p2:])
    return out

remix_filtered = creative_filter(remix, sr1)
remix_filtered /= np.max(np.abs(remix_filtered)) * 1.01 # Normalize peak just below 1

# ===== SAVE OUTPUT =====
sf.write('remix_sad_to_happy.wav', remix_filtered, sr1)
print("\nRemix saved as remix_sad_to_happy.wav")

# ===== VISUALIZATION =====
plt.figure(figsize=(15, 9))
plt.subplot(3,1,1)
librosa.display.waveshow(y1, sr=sr1, color='blue')
plt.title("Waveform Original Lagu 1 (Sad)")
plt.subplot(3,1,2)
librosa.display.waveshow(y2, sr=sr2, color='orange')
plt.title("Waveform Original Lagu 2 (Happy)")

plt.figure(figsize=(15, 9))
plt.subplot(3,1,1)
D1 = librosa.amplitude_to_db(np.abs(librosa.stft(y1)), ref=np.max)
librosa.display.specshow(D1, sr=sr1, x_axis='time', y_axis='hz')
plt.title("Spectrogram - Lagu 1 (Sad)")
plt.colorbar(format='%+2.0f dB')
plt.subplot(3,1,2)
D2 = librosa.amplitude_to_db(np.abs(librosa.stft(y2)), ref=np.max)
librosa.display.specshow(D2, sr=sr2, x_axis='time', y_axis='hz')
plt.title("Spectrogram - Lagu 2 (Happy)")
plt.colorbar(format='%+2.0f dB')

```

C:\Users\Useradmin\AppData\Local\Temp\ipykernel_5540\497829401.py:11: DeprecationWarning: Conversion of an array with ndim > 0 to a scalar is deprecated, and will error in future. Ensure you extract a single element from your array before performing this operation. (Deprecated NumPy 1.25.)

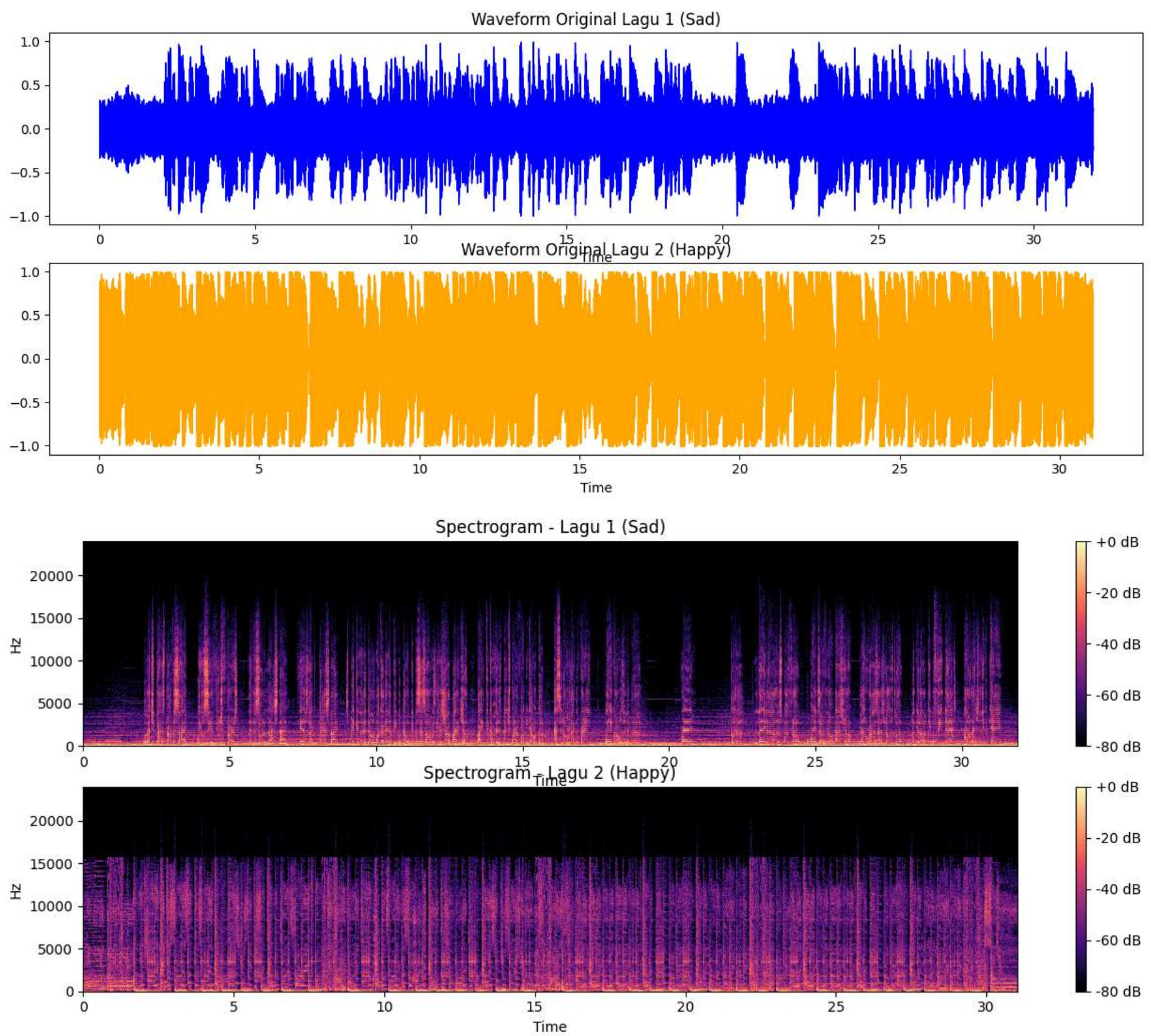
tempo = float(tempo)

Lagu 1 (Sad): BPM=100.4, Key=C

Lagu 2 (Happy): BPM=133.9, Key=F

Remix saved as remix_sad_to_happy.wav

Out[15]: <matplotlib.colorbar.Colorbar at 0x1dd1ba41450>

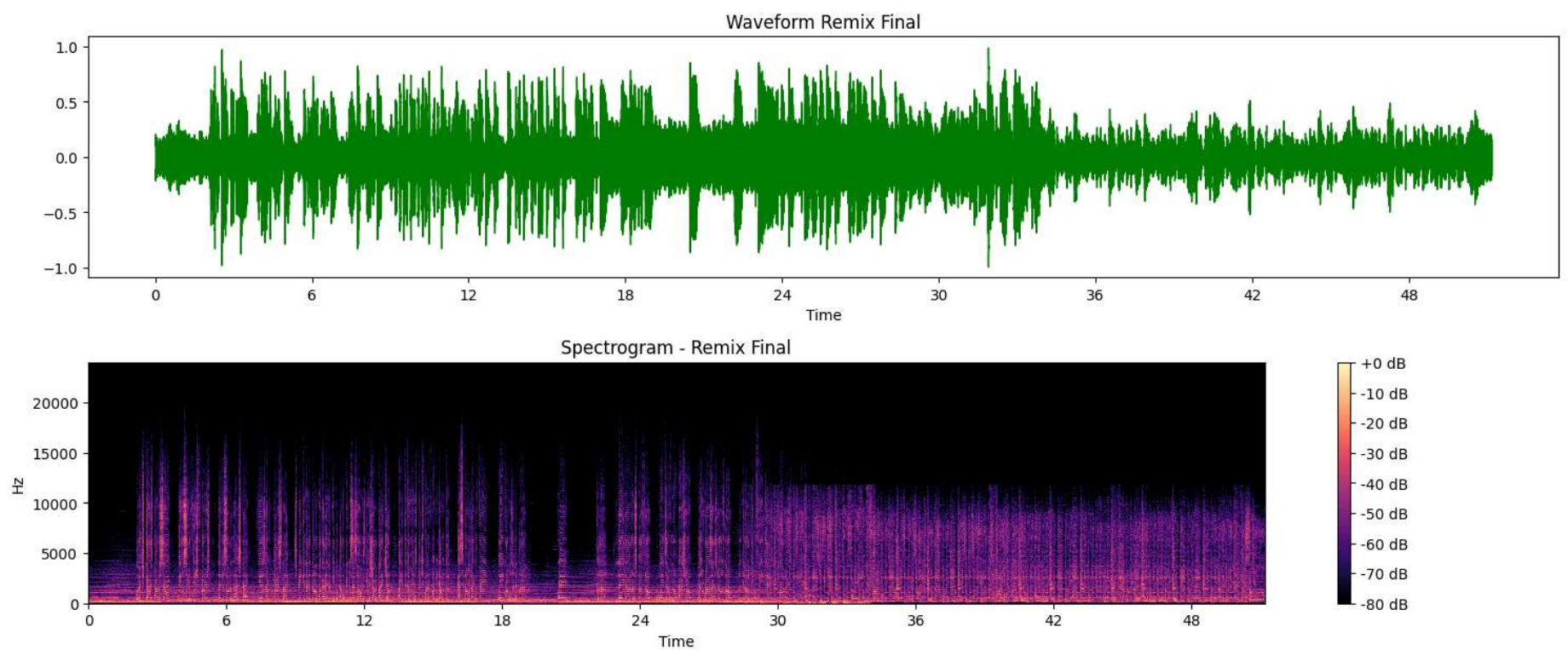


Jelaskan proses dan parameter yang digunakan:

Proses pertama pada analisis audio adalah memahami karakteristik musikal dari kedua lagu dengan memahami berapa BPM dan kunci nadanya (y = raw data sr = sample rate). Kemudian digunakan time-stretching agar tidak terkesan terlalu memaksakan ketika transisi dari Sad.wav ke Happy.wav. Kunci nada lagu kedua yang sudah disesuaikan temponya diubah agar cocok dengan kunci nada lagu pertama ($y2_stretched$, n_steps), jika kedua lagu sudah cocok, keduanya digabungkan dengan teknik crossfade untuk transisi yang mulus ($y1$, $y2$, $duration_sec$). penambahan filter dengan menggunakan paramater yang sudah ada seperti low-pass, high-pass, dan band-pas. `butter(N, Wn, ...)`

- Tampilkan waveform dan spektrogram sesudah remix.

```
In [16]: plt.figure(figsize=(15, 9))
plt.subplot(3,1,1)
librosa.display.waveshow(remix_filtered, sr=sr1, color='green')
plt.title("Waveform Remix Final")
plt.subplot(3,1,2)
D3 = librosa.amplitude_to_db(np.abs(librosa.stft(remix_filtered)), ref=np.max)
librosa.display.specshow(D3, sr=sr1, x_axis='time', y_axis='hz')
plt.title("Spectrogram - Remix Final")
plt.colorbar(format='%+2.0f dB')
plt.tight_layout()
plt.show()
```

Jelaskan hasil remix yang telah dilakukan:

Pertama dilakukan penyelarasan nada terlebih dahulu agar lagu kedua dibuat sama dengan lagu pertama, cross fade untuk transisi lebih halus saat pergantian lagu, efek dari filter yang ditambahkan

Prompt yang dikirimkan ke CHATGPT: "• Lakukan pitch shifting pada rekaman suara Soal 1 untuk membuat suara terdengar seperti chipmunk (dengan mengubah pitch ke atas). • Visualisasikan waveform dan spektrogram sebelum dan sesudah pitch shifting. • Jelaskan proses pitch shifting yang Anda lakukan, termasuk: – Parameter yang digunakan – Perbedaan dalam representasi visual antara suara asli dan suara yang telah dimodifikasi – Bagaimana perubahan pitch memengaruhi kualitas dan kejelasan suara • Gunakan dua buah pitch tinggi, misalnya pitch +7 dan pitch +12. • Gabungkan kedua rekaman yang telah di-pitch shift ke dalam satu file audio. (Gunakan ChatGPT / AI untuk membantu Anda dalam proses ini)"

breakdown this step by step

Soal 5: Music Analysis dan Remix • Pilih 2 buah (potongan) lagu yang memiliki vokal (penyanyi) dan berdurasi sekitar 1 menit: – Lagu 1: Nuansa sedih, lambat – Lagu 2: Nuansa ceria, cepat • Konversikan ke format WAV sebelum dimuat ke notebook. • Lakukan deteksi tempo (BPM) dan estimasi kunci (key) dari masing-masing lagu dan berikan analisis singkat. • Lakukan remix terhadap kedua lagu: – Time Stretch: Samakan tempo kedua lagu – Pitch Shift: Samakan kunci (key) kedua lagu – Crossfading: Gabungkan kedua lagu dengan efek crossfading – Filter Tambahan: Tambahkan filter kreatif sesuai keinginan (opsional) • Jelaskan proses dan parameter yang digunakan. • Tampilkan waveform dan spektrogram sesudah remix. • Jelaskan hasil remix yang telah dilakukan.

same with this one, is the same like nomer 3?